

РО 1.4. Основные сервисы Linux для предприятия

Лекция 7.

OpenSSL, собственный CA и TLS-сертификаты для серверных служб

Цель лекции

- Понять, какие угрозы возникают при передаче данных без шифрования
- Объяснять назначение TLS, цифрового сертификата, закрытого ключа, CSR и CA
- Различать Root CA, subordinate CA, серверный сертификат и цепочку доверия
- Выполнять базовый порядок выпуска сертификата через OpenSSL
- Проверять сертификат локально, по сети и по типовым ошибкам клиента

Учебные вопросы

1. Проблема безопасности данных в сети и основы криптографии
2. Инфраструктура открытых ключей PKI и роль удостоверяющего центра CA
3. Жизненный цикл TLS-сертификата и процесс DER
4. Инструмент OpenSSL в Linux
5. Практика: создание собственного CA предприятия
6. Практика: выпуск TLS-сертификата для веб-сервера
7. Публикация и настройка TLS в серверных службах
8. Диагностика, проверка и типовые ошибки

Учебный сценарий лекции

Внутренняя учебная зона организации: company.test
СА-сервер ca01.company.test выпускает сертификаты
для внутренних служб

Веб-сервер web01.company.test имеет адрес
192.168.10.20 и обслуживает HTTPS

Клиент client01 должен доверять корневому
сертификату компании и открывать web01 без ошибки
доверия

1. Проблема безопасности данных в сети и основы криптографии

Если служба передает логины, пароли и письма открытым текстом, их можно перехватить в сети

HTTP, LDAP, SMTP и IMAP без TLS не защищают веб, каталог, отправку и получение почты от просмотра

Администратор должен защищать не только публичные сайты, но и внутренние сервисы предприятия

TLS (Transport Layer Security) — протокол защиты соединения, который обеспечивает шифрование и проверку подлинности сервера

Что именно защищает TLS

Конфиденциальность означает, что посторонний наблюдатель не может прочесть передаваемые данные

Целостность означает, что клиент и сервер обнаружат изменение данных в пути

Подлинность сервера означает, что клиент понимает, с каким сервером он установил соединение

TLS не исправляет слабые пароли и ошибки доступа, но защищает канал передачи данных

Симметричное шифрование

Симметричное шифрование использует один общий ключ для шифрования и расшифрования данных

Такой подход быстрый и подходит для передачи больших объемов трафика

Главная проблема: клиент и сервер должны безопасно договориться об общем ключе

Если общий ключ перехвачен, защищенность всего соединения теряется

Асимметричное шифрование

Асимметричное шифрование использует пару ключей: public key и private key

Public key, открытый ключ, можно передавать клиентам и включать в сертификат

Private key, закрытый ключ, хранится только у владельца сервера или СА

Закрытый ключ нельзя отправлять по сети, копировать в отчеты или размещать в общих папках

Цифровая подпись и хэш

Hash, хэш, является коротким цифровым отпечатком данных, который меняется при изменении исходного текста

Digital Signature, цифровая подпись, подтверждает, что данные подписаны владельцем закрытого ключа

CA подписывает сертификат своим закрытым ключом, а клиент проверяет подпись открытым ключом CA

Подпись не скрывает данные, а подтверждает источник и целостность

Шифрование, подпись и доверие в TLS

Асимметричное шифрование

Сервер генерирует пару ключей:

Public Key

Шифрует данные и проверяет подпись

Private Key

Расшифровывает данные и создаёт подпись

- Public Key передаётся в сертификате
- Private Key — строго конфиденциален
- Подпись = хэш данных, зашифрованный Private Key
- Проверка подписи — Public Key

TLS Handshake

- 1 Клиент → Сервер: запрос
- 2 Сервер → Клиент: сертификат + Public Key
- 3 Клиент проверяет подпись через доверенный CA
- 4 Клиент шифрует сессионный ключ Public Key сервера
- 5 Далее — симметричное шифрование (быстро, надёжно)

Цепочка доверия (PKI)



Корневой CA

Самоподписанный, доверенный



Subordinate CA

Выпускает серверные сертификаты



Серверный сертификат

Содержит Public Key + SAN



Клиент проверяет

подпись по цепочке до корня

 **TLS = асимметричное шифрование (установка сессии) + симметричное (данные) + цифровая подпись (подлинность сервера)**

Без PKI и доверенного CA невозможна проверка подлинности сервера → риск атаки Man-in-the-Middle



2. Инфраструктура открытых ключей PKI и роль удостоверяющего центра СА

PKI (Public Key Infrastructure) — инфраструктура открытых ключей, которая управляет сертификатами и доверием

Без PKI клиент видит открытый ключ сервера, но не знает, можно ли этому ключу доверять

MitM (Man in the Middle) — атака человека посередине, когда злоумышленник подменяет соединение между клиентом и сервером

СА решает задачу доверия: он подтверждает, что сертификат действительно принадлежит нужному владельцу

Х.509-сертификат

Х.509 — распространенный стандарт цифровых сертификатов для TLS и других PKI-сценариев

Certificate, сертификат, связывает имя владельца с его открытым ключом и данными издателя

Сертификат содержит срок действия, серийный номер, алгоритм подписи и назначение ключа

Сертификат сервера не должен содержать закрытый ключ

SAN и имена сервера

SAN (Subject Alternative Name) — расширение сертификата со списком DNS-имен и IP-адресов сервера

Common Name, общее имя субъекта сертификата, больше не является достаточным способом проверки имени

Современные браузеры проверяют имя сервера по SAN, а не только по Common Name

Для web01.company.test в SAN нужно указать
DNS:web01.company.test

Если клиент заходит по IP-адресу 192.168.10.20, IP тоже должен быть указан в SAN

Certificate Authority

CA (Certificate Authority) — удостоверяющий центр, который выпускает и подписывает сертификаты

CA проверяет запрос, подписывает сертификат и тем самым связывает открытый ключ с владельцем

В учебной сети собственный CA позволяет защищать внутренние веб, почтовые и LDAP-службы

Доверие к сертификатам внутреннего CA работает только после установки корневого сертификата на клиенты

Root CA и subordinate CA

Root CA, корневой удостоверяющий центр, является верхней точкой доверия в PKI

Закрытый ключ Root CA должен храниться особенно строго, потому что компрометация ломает всю цепочку доверия

Subordinate CA, подчиненный удостоверяющий центр, получает сертификат от Root CA и выпускает серверные сертификаты

В учебной лаборатории можно начать с Root CA, но в крупных системах выпуск часто делегируют subordinate CA

Цепочка доверия

Chain of Trust, цепочка доверия, связывает серверный сертификат с доверенным корневым сертификатом

Клиент проверяет подпись серверного сертификата и подписи промежуточных сертификатов

Если корневой сертификат СА есть в доверенном хранилище клиента, цепочка считается доверенной

Если корневого СА нет, браузер или утилита покажет ошибку неизвестного издателя



Root CA, Subordinate CA и серверный сертификат

Иерархия доверия в PKI: от корневого центра сертификации до сервера.



Идея проста: корень доверяют все, субординированный CA выпускает сертификаты, а серверный сертификат защищает соединение с сервисом.

1. ROOT CA (КОРНЕВОЙ CA)



Самый верхний уровень доверия. Подписывает subordinate CA.



Хранит закрытый ключ офлайн в максимально защищённом месте.



Сертификат Root CA устанавливается в доверенные хранилища клиентов.



Доверие к Root CA = доверие ко всей цепочке ниже.

Пример

corprootCA.company.test
(срок действия 10-20 лет)

2. SUBORDINATE CA (ПОДЧИНЁННЫЙ CA)



Подписан Root CA. Выпускает сертификаты для серверов, клиентов и других служб.



Хранит закрытый ключ онлайн на защищённом CA-сервере.



Его сертификат не доверяют напрямую — доверяют через Root CA.



Можно создавать несколько subordinate CA для разных отделов и задач.

Пример

corp-issuing-CA.company.test
(срок действия 3-5 лет)

3. СЕРВЕРНЫЙ СЕРТИФИКАТ



Подписан subordinate CA. Используется конкретным сервером для TLS (HTTPS, LDAPS и т.д.).



Закрытый ключ хранится на сервере службы (web, mail, vpn и др.).



Содержит имена (SAN), IP-адреса и назначение ключа (серверная аутентификация).

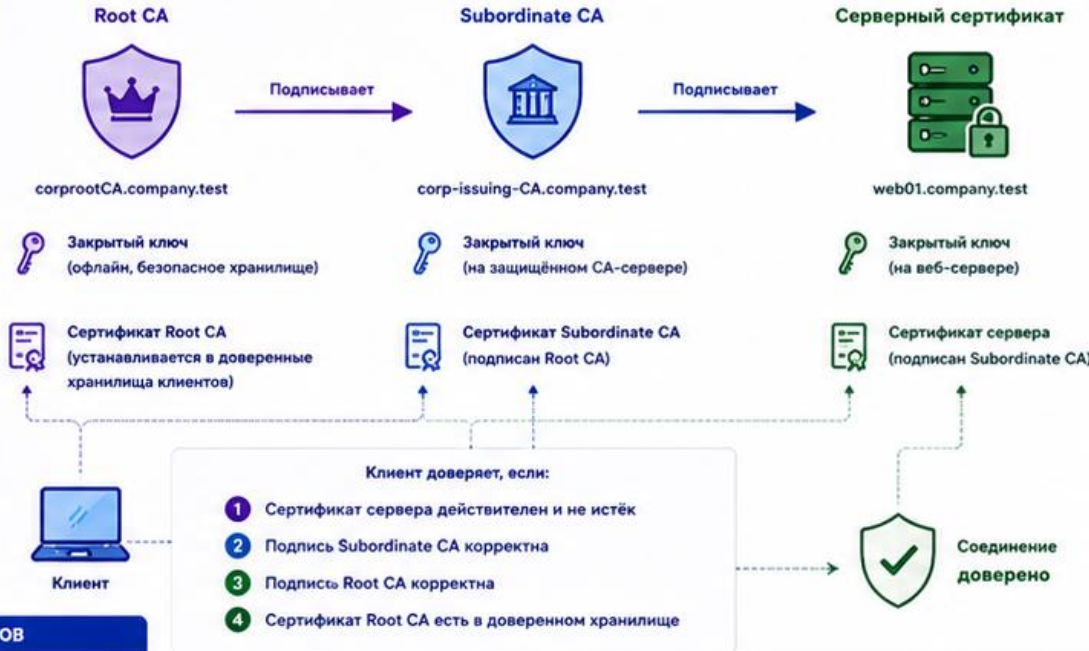


Если срок содействия истёк — соединение будет отвергнуто клиентом.

Пример

web01.company.test
(срок действия 1-5 лет)

ЦЕПОЧКА ДОВЕРИЯ В PKI



СРАВНЕНИЕ УРОВНЕЙ

Параметр	Root CA	Subordinate CA	Серверный сертификат
Назначение	Создаёт доверие для всей PKI	Выпускает сертификаты для служб и пользователей	Защищает конкретный сервис
Кем подписан	Самоподписан	Подписан Root CA	Подписан Subordinate CA
Закрытый ключ	Офлайн, строго защищён	Онлайн на CA-сервере, защищён	На сервере службы, доступен приложению
Кем доверяют	Устанавливается вручную в хранилища	Доверяют через Root CA	Доверяют через Subordinate CA и Root CA
Срок действия	10-20 лет	2-5 лет	6-24 месяца
Пример имени	corprootCA.company.test	corp-issuing-CA.company.test	web01.company.test

ХАРАКТЕРИСТИКИ СЕРТИФИКАТОВ

- Subject (CN) — общее имя владельца
- SAN — альтернативные имена (DNS, IP)
- Issuer — кто выдал сертификат
- Serial Number — серийный номер
- Not Before / Not After — срок действия
- Key Usage — назначение ключа
- Extended Key Usage — расширенное назначение

ЛУЧШИЕ ПРАКТИКИ

- ✓ Храните Root CA офлайн и делайте резервные копии в сейфе.
- ✓ Используйте отдельный Subordinate CA для ежедневных операций.
- ✓ Ограничивайте права доступа к CA и закрытым ключам.
- ✓ Выпускайте сертификаты только на необходимые имена и службы.
- ✓ Регулярно проверяйте срок действия и продлевайте вовремя.
- ✓ Отзывайте компрометированные сертификаты (CRL/OCSP).
- ✓ Документируйте процессы выпуска и хранения ключей.



ИТОГ:

Root CA создаёт доверие. Subordinate CA выпускает. Серверный сертификат защищает. Правильная иерархия и хранение ключей — основа безопасной инфраструктуры.



ВАЖНО:

Если любой сертификат в цепочке недействителен, клиент не доверит соединение. Следите за сроками, хранилищами и корректной цепочкой!

Доверенное хранилище клиентов

Trust Store, доверенное хранилище, содержит корневые сертификаты CA, которым доверяет система или браузер

На Linux-клиенте CA обычно добавляют в `/usr/local/share/ca-certificates` и запускают `update-ca-certificates`

На Windows и в некоторых браузерах есть отдельные хранилища доверенных корневых центров

Один и тот же сертификат сервера может быть доверенным на одном клиенте и недоверенным на другом

3. Жизненный цикл TLS-сертификата и процесс DERA

TLS-сертификат не появляется сам: администратор проходит последовательность подготовки ключа, запроса, подписи и установки

DERA используется здесь как учебная памятка, а не как официальный отраслевой стандарт

- D — private key: сгенерировать закрытый ключ сервера
- E — CSR: создать запрос на подпись сертификата
- R — CA signing: подписать запрос удостоверяющим центром
- A — installation: установить сертификат на службу и проверить клиента

D: закрытый ключ сервера

Закрытый ключ `web01.key` создается на сервере или в защищенной рабочей среде администратора

Ключ сервера нужен Nginx или Apache для TLS-рукопожатия

Для автоматического старта службы серверный ключ часто делают без парольной фразы

Если `web01.key` утечет, сертификат нужно перевыпустить, а старый отозвать или заменить

Е: запрос CSR

CSR (Certificate Signing Request) — запрос на подпись сертификата

CSR содержит открытый ключ сервера и сведения, которые CA должен включить в сертификат

CSR можно передавать CA, потому что он не содержит закрытый ключ

В CSR или отдельном файле расширений обязательно задают SAN для DNS-имен и IP-адресов

R: подписание запроса CA

CA проверяет CSR и выпускает сертификат web01.crt

Сертификат подписывается закрытым ключом CA,
например ca.key

Серийный номер помогает различать выпущенные
сертификаты и вести учет

Срок действия серверного сертификата обычно
короче срока действия корневого CA

А: установка и проверка

Сертификат web01.crt и ключ web01.key
устанавливают в конфигурацию веб-сервера

Клиент должен доверять СА, который подписал
web01.crt

Администратор проверяет совпадение имени
сервера, SAN, срока действия и цепочки доверия

После проверки HTTPS должен открываться без
предупреждения о недоверенном сертификате



ЖИЗНЕННЫЙ ЦИКЛ СЕРТИФИКАТА (DERA)

DERA — это стандартный цикл управления сертификатом от запроса до отзыва.



DERA помогает обеспечить безопасность: сертификат выдан только тому, кому доверяем, и используется только пока он актуален.

1 D - DISCOVERY

ОПРЕДЕЛЕНИЕ ПОТРЕБНОСТИ



Выявляется потребность в сертификате.

- Используемые сервисы
- Тип сертификата
- Требуемые имена (SAN)
- Срок действия
- Политики и требования

2 E - ENROLLMENT

ЗАПРОС (CSR)



Администратор формирует запрос на сертификат (CSR).

- Генерация ключевой пары
- Создание CSR
- Передача CSR в CA
- Подтверждение владения именами (если требуется)

3 R - REVIEW

ПРОВЕРКА И ВЫПУСК



Удостоверяющий центр (CA) проверяет запрос и выпускает сертификат.

- Проверка данных заявителя
- Проверка владения именами
- Подпись сертификата CA
- Публикация в репозитории (если настроено)

4 A - ARRIVAL

ПОЛУЧЕНИЕ И УСТАНОВКА



Сертификат передаётся владельцу и устанавливается на сервер/клиент.

- Получение сертификата и цепочки
- Установка закрытого ключа
- Установка сертификата и цепочки CA
- Настройка сервиса для использования сертификата

5 R - RENEWAL

ПРОДЛЕНИЕ



Сертификат продлевается до истечения срока действия.

- Мониторинг срока действия
- Создание нового CSR
- Выпуск нового сертификата
- Установка нового сертификата и повторное тестирование

6 A - REVOCATION

ОТЗЫВ



При компрометации или выводе из эксплуатации сертификат отзывается.

- Запрос на отзыв (CRL/OCSP)
- Публикация информации об отзыве
- Удаление сертификата с серверов и клиентов
- Замена при необходимости

DERA В ДЕЙСТВИИ (ПРИМЕР ДЛЯ ВЕБ-СЕРВЕРА)

Discovery

Планируем HTTPS для web01.company.test



Enrollment

Создаём ключ и CSR



Review

CA проверяет и подписывает



Arrival

Устанавливаем сертификат



Renewal

Обновляем до истечения срока действия



Revocation

Отзываем при компрометации



Главная цель DERA: сертификат правильного владельца, для правильных имён, в нужный срок и с возможностью быстрой замены или отзыва.

ЧТО УЧАСТВУЕТ В ЖИЗНЕННОМ ЦИКЛЕ



Владелец сертификата

Запрашивает, получает и устанавливает сертификат для своих систем и сервисов.



Удостоверяющий центр (CA)

Проверяет запрос, подписывает сертификат и управляет его жизненным циклом.



Репозиторий / CRL / OCSP

Публикует информацию о статусе сертификатов для проверки клиентами.



Клиенты

Проверяют цепочку доверия и статус сертификата при каждом соединении.

КОРОТКО О КАЖДОМ ЭТАПЕ

- D - Discovery** Планирование и определение требований: какие имена, какой срок, где используется.
- E - Enrollment** Генерация ключа и CSR, подтверждение владения именами (HTTP/почта/DNS и др.).
- R - Review** Проверка запроса CA, подпись и выпуск сертификата, публикация (CRL/OCSP).
- A - Arrival** Доставка сертификата владельцу, установка на сервер/клиент, настройка службы.
- R - Renewal** Заблаговременное продление: новый CSR, новый сертификат, тестирование и замена.
- A - Revocation** Отзыв при компрометации/увольнении/смене ключа, публикация статуса, замена.

ЛУЧШИЕ ПРАКТИКИ

- ✓ Используйте ключи подходящей длины (RSA 2048+, ECDSA P-256+).
- ✓ Храните закрытые ключи безопасно (права доступа, шифрование).
- ✓ Устанавливайте только нужные имена в SAN.
- ✓ Следите за сроками и начинайте продление заранее.
- ✓ Включайте OCSP (или проверку CRL) на клиентах и серверах.
- ✓ Документируйте процессы выдачи, продления и отзыва.

ТИПОВЫЕ ПРОБЛЕМЫ

- ⚠ Истёк срок действия сертификата.
- ⚠ Имя сервера не совпадает с SAN.
- ⚠ Неполная цепочка сертификатов на сервере.
- ⚠ Клиент не доверяет Root CA (не установлен корневой сертификат).
- ⚠ Сертификат отозван (CRL/OCSP показывает статус revoked).
- ⚠ Закрытый ключ утрачен или скомпрометирован.



Карта файлов сертификата

- `ca.key` — закрытый ключ корневого СА; самый чувствительный файл учебной PKI
- `ca.crt` — корневой сертификат СА, который устанавливается клиентам как доверенный
- `web01.key` — закрытый ключ веб-сервера, который хранится только на `web01`
- `web01.csr` — запрос на подпись, который СА использует для выпуска сертификата
- `web01.crt` — сертификат веб-сервера, который публикуется в настройке HTTPS

4. Инструмент OpenSSL в Linux

OpenSSL — набор криптографических библиотек и консольная утилита для работы с ключами и сертификатами

В Linux OpenSSL часто используется для генерации ключей, CSR, сертификатов и диагностики TLS

Утилита `openssl` состоит из подкоманд, например `genrsa`, `req`, `x509` и `s_client`

Команды OpenSSL нужно выполнять внимательно: ошибка в имени, SAN или сроке действия сразу попадет в сертификат

openssl genrsa

RSA (Rivest-Shamir-Adleman) — алгоритм асимметричной криптографии, используемый для ключей и подписей

openssl genrsa генерирует RSA-закрытый ключ

Пример: openssl genrsa -out web01.key 2048 создает ключ веб-сервера длиной 2048 бит

Для СА лучше использовать более длинный ключ, например 4096 бит

Результат команды — файл ключа; его нужно сразу защитить правами доступа

openssl req

openssl req используется для создания CSR и самоподписанных сертификатов

Пример: openssl req -new -key web01.key -out web01.csr -subj /CN=web01.company.test

Параметр -new создает новый запрос, -key указывает закрытый ключ, -out задает файл CSR

Параметр -subj задает subject без интерактивного ввода, но SAN нужно задавать отдельно

openssl x509

openssl x509 просматривает, проверяет и выпускает X.509-сертификаты

Пример: `openssl x509 -in web01.crt -text -noout`
показывает содержимое сертификата

Параметр `-text` выводит поля сертификата, а `-noout` скрывает PEM-блок

Для подписания CSR команда x509 используется вместе с `-req`, `-CA`, `-CAkey` и `-extfile`

Форматы файлов сертификатов

PEM (Privacy Enhanced Mail) — текстовый формат с блоками BEGIN и END, удобный для Linux-служб

- .key обычно содержит закрытый ключ в PEM-формате

- .csr содержит запрос на подпись сертификата

- .crt или .pem часто содержит сертификат в текстовом PEM-формате

DER (Distinguished Encoding Rules) — бинарный формат сертификата

- .p12 или .pfx обычно означает PKCS#12-контейнер с сертификатом и закрытым ключом

5. Практика: создание собственного СА предприятия

Собственный СА нужен для внутренних сервисов, которые не должны получать публичные сертификаты

СА создают на отдельном защищенном узле или в отдельной защищенной директории

В учебном сценарии СА размещается на `ca01.company.test` в каталоге `/root/ca`

Перед началом важно сделать снимок ВМ и не использовать учебные ключи в реальной сети

Структура каталогов СА

- `mkdir -p /root/ca/private /root/ca/certs /root/ca/csr`
создает базовые каталоги СА
- Каталог `private` хранит закрытый ключ СА и должен быть доступен только администратору
- Каталог `certs` хранит выпущенные сертификаты
- Каталог `csr` хранит запросы на подпись, которые приходят от серверов

Генерация закрытого ключа СА

`openssl genrsa -aes256 -out /root/ca/private/ca.key 4096` создает закрытый ключ СА с парольной фразой

Параметр `-aes256` шифрует ключ на диске и снижает риск использования ключа после кражи файла `chmod 400 /root/ca/private/ca.key` оставляет доступ к ключу только владельцу

Потеря `ca.key` означает невозможность нормально выпускать и проверять новые сертификаты этой PKI

Создание корневого сертификата CA

`openssl req -x509 -new -key /root/ca/private/ca.key -sha256 -days 3650 -out /root/ca/certs/ca.crt -subj /CN=Company Test Root CA` выпускает корневой сертификат

Параметр `-x509` создает самоподписанный сертификат, а не обычный CSR

Параметр `-days 3650` задает срок действия около 10 лет для учебного корневого CA

Файл `ca.crt` можно распространять клиентам, но `ca.key` остается только на CA-сервере

openssl.cnf для автоматизации

openssl.cnf позволяет заранее описать параметры выпуска сертификатов и расширения

В конфигурации удобно задавать keyUsage, extendedKeyUsage и subjectAltName

Автоматизация снижает риск забыть SAN или выпустить сертификат с неправильным назначением

Даже при автоматизации администратор проверяет итоговый сертификат через openssl x509

ЗАЩИЩЕННАЯ СТРУКТУРА СОБСТВЕННОГО СА

Надежный центр доверия для внутренней инфраструктуры



ДОВЕРИЕ

Единый источник доверия в организации



БЕЗОПАСНОСТЬ

Защита закрытых ключей и процессов СА



КОНТРОЛЬ

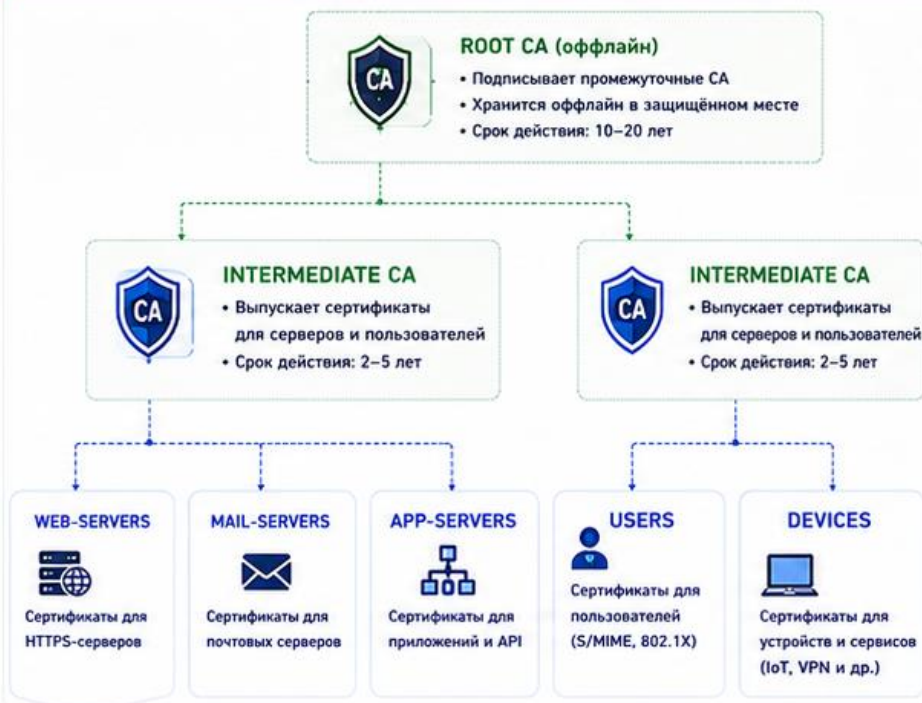
Политики, аудит и управление доступом



НЕПРЕРЫВНОСТЬ

Резервирование и план восстановления

1. ЛОГИЧЕСКАЯ СТРУКТУРА СОБСТВЕННОГО СА



2. РОЛИ И СЕРВЕРЫ



3. ПОТОК ВЫПУСКА СЕРТИФИКАТА



4. ЗАЩИТА СА



5. ПУБЛИКАЦИЯ И ПРОВЕРКА ДОВЕРИЯ



6. РЕЗЕРВИРОВАНИЕ И ВОССТАНОВЛЕНИЕ



7. РЕКОМЕНДУЕМЫЕ ПРАКТИКИ

- ✓ Разделяйте роли: Root CA оффлайн
- ✓ Используйте Intermediate CA для ежедневной работы
- ✓ Храните ключи в HSM или оффлайн
- ✓ Включайте CRL/OCSP и короткие сроки действия
- ✓ Ограничивайте доступ и ведите аудит
- ✓ Мониторьте и уведомляйте об инцидентах
- ✓ Регулярно тестируйте восстановление
- ✓ Документируйте процессы и политики

ТРЕБОВАНИЯ К СА-СЕРВЕРАМ

- Минимум сервисов и обновлённая ОС
- Синхронизация времени (NTP)
- Надёжное хранение ключей (HSM/оффлайн)

МИНИМАЛЬНЫЕ ПОРТЫ

- LDAP/LDAPS: 389 / 636
- HTTP/HTTPS (для публикации): 80 / 443
- OCSP (при наличии): 80 / 443

РЕКОМЕНДУЕМЫЕ СРОКИ

- Root CA: 10–20 лет
- Intermediate CA: 2–5 лет
- Серверные сертификаты: до 1 года



ИТОГ

Собственный СА — это фундамент доверия. Правильная архитектура, защита ключей и процессы обеспечивают безопасность всей инфраструктуры.

6. Практика: выпуск TLS-сертификата для веб-сервера

Веб-сервер `web01.company.test` должен получить собственный закрытый ключ и сертификат

Закрытый ключ сервера создается отдельно от ключа CA

CSR передается на CA для подписи и не содержит секретных данных

CA выпускает `web01.crt`, который затем устанавливается на веб-сервер

Генерация ключа web01

`openssl genrsa -out web01.key 2048` создает RSA-ключ веб-сервера

Ключ без парольной фразы позволяет Nginx автоматически запускаться после перезагрузки

Такой ключ нужно защищать правами файловой системы и доступом к серверу

Если ключ создан не на web01, его нужно передавать только по защищенному каналу

Файл расширений SAN

Файл web01.ext нужен, чтобы добавить в сертификат DNS-имя и IP-адрес сервера

Пример строки: subjectAltName =
DNS:web01.company.test, IP:192.168.10.20

Если SAN отсутствует, браузер может отвергнуть сертификат даже при совпадающем Common Name

SAN должен соответствовать тому имени или адресу, по которому клиент реально подключается к серверу

Создание CSR для web01

`openssl req -new -key web01.key -out web01.csr -subj /CN=web01.company.test` создает CSR

CSR связывает открытый ключ сервера с именем `web01.company.test`

Администратор передает `web01.csr` на CA-сервер для подписи

После создания CSR полезно проверить, что CN и публичный ключ соответствуют ожидаемому серверу

Подписание CSR на СА

```
openssl x509 -req -in web01.csr -CA ca.crt -CAkey ca.key  
-CAcreateserial -out web01.crt -days 365 -sha256 -extfile  
web01.ext
```

 подписывает запрос

Параметр -CA указывает сертификат СА, а -CAkey указывает закрытый ключ СА

Параметр -CAcreateserial создает файл с серийным номером, если его еще нет

Параметр -extfile добавляет SAN и другие расширения из файла web01.ext

Проверка выпущенного сертификата

`openssl x509 -in web01.crt -text -noout` показывает поля сертификата

В выводе нужно проверить Issuer, Subject, Validity и X509v3 Subject Alternative Name

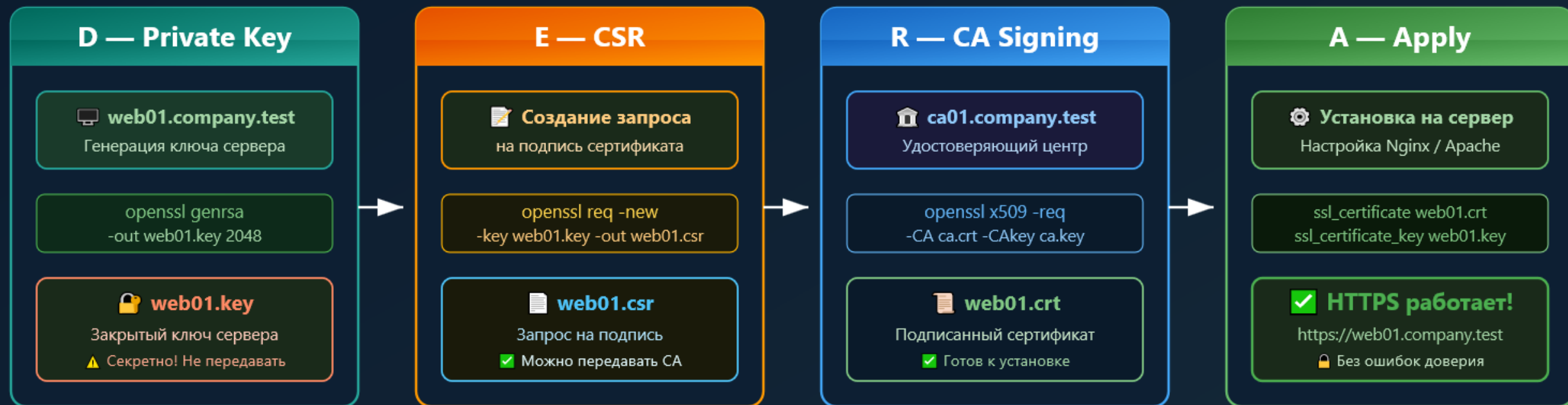
`openssl verify -CAfile ca.crt web01.crt` проверяет, доверяет ли сертификат заданному СА

Ожидаемый результат `verify: web01.crt: OK`

Выпуск сертификата веб-сервера через СА

16:9

Процесс DERA (Generate key → Create CSR → Sign with CA → Apply to service)



📁 Карта файлов сертификата		
Файл	Содержит	Статус
ca.key	Закрытый ключ СА	🔒 Секретно
ca.crt	Корневой сертификат СА	👤 Для клиентов
web01.key	Закрытый ключ сервера	🔒 Секретно
web01.crt	Сертификат сервера	👤 Публичный

Проверка сертификата

```
openssl x509 -in web01.crt -text -noout
→ Просмотр полей сертификата
```

```
openssl verify -CAfile ca.crt web01.crt
→ Ожидаемый результат: OK
```

```
openssl s_client -connect web01:443 -CAfile ca.crt
→ Verify return code: 0 (ok)
```

7. Публикация и настройка TLS в серверных службах

После выпуска сертификата серверная служба должна знать путь к сертификату и закрытому ключу

Для Nginx сертификат обычно размещают в /etc/ssl/certs, а закрытый ключ в /etc/ssl/private

Служба должна читать закрытый ключ, но обычные пользователи не должны иметь к нему доступ

После изменения конфигурации нужно проверить синтаксис и перезагрузить службу

Фрагмент настройки Nginx HTTPS

`listen 443 ssl` включает HTTPS-порт для виртуального сервера

`server_name web01.company.test` должен совпадать с DNS-именем в SAN сертификата

`ssl_certificate /etc/ssl/certs/web01.crt` указывает серверный сертификат

`ssl_certificate_key /etc/ssl/private/web01.key` указывает закрытый ключ сервера

`ssl_protocols TLSv1.2 TLSv1.3` ограничивает работу современными версиями TLS

Права доступа к закрытому ключу сервера

```
sudo chown root:root /etc/ssl/private/web01.key
```

назначает владельцем ключа root

```
sudo chmod 600 /etc/ssl/private/web01.key
```

 разрешает чтение и запись только владельцу

Для некоторых схем доступа используют группу `ssl-cert` и режим `640`, если это требуется службе

Ошибка `Permission denied` в логах `Nginx` часто означает, что служба не может прочитать ключ

Проверка и перезагрузка Nginx

`sudo nginx -t` проверяет синтаксис конфигурации
Nginx до перезагрузки

`sudo systemctl reload nginx` применяет настройки без
полной остановки службы

`sudo ss -tulpen | grep ':443'` проверяет, слушает ли
сервер HTTPS-порт

Если порт 443 не слушается, клиент физически не
сможет установить TLS-соединение

Импорт CA на Linux-клиенте

`sudo cp ca.crt /usr/local/share/ca-certificates/company-test-root-ca.crt` копирует CA в системный каталог доверия

`sudo update-ca-certificates` обновляет доверенное хранилище сертификатов Linux

После импорта клиентские программы смогут доверять сертификатам, подписанным Company Test Root CA

Если приложение использует собственное хранилище сертификатов, системного импорта может быть недостаточно

ПУБЛИКАЦИЯ HTTPS-СЕРВИСА И ДОВЕРИЕ КЛИЕНТА

От установки сертификата на сервер до зелёного замка в браузере



ШИФРОВАНИЕ
Данные защищены от перехвата



ПОДЛИННОСТЬ
Клиент уверен, что это нужный сервер



ЦЕЛОСТНОСТЬ
Данные не изменены в пути



ДОВЕРИЕ
Сертификат подтверждён доверенной СА

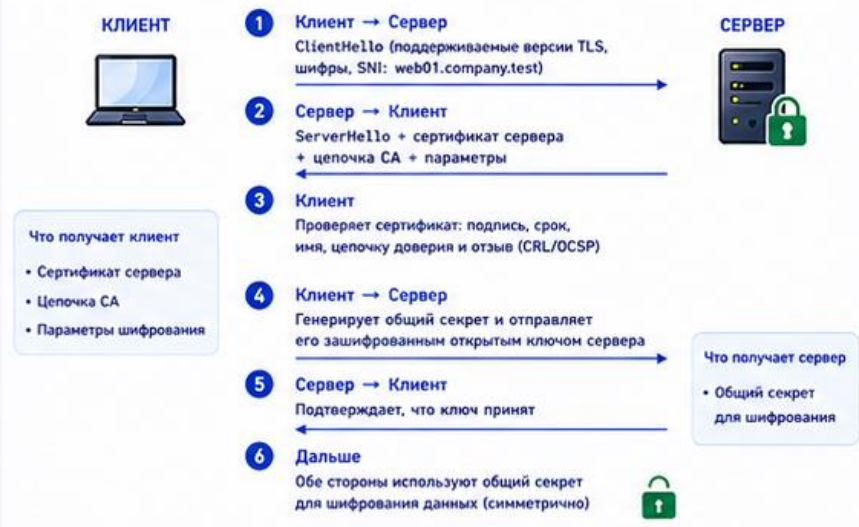
1. КОМПОНЕНТЫ СХЕМЫ



Что получаем

- ✓ Конфиденциальность — никто в сети не читает данные
- ✓ Подлинность — клиент уверен, что подключён к web01.company.test
- ✓ Целостность — изменение данных будет обнаружено
- ✓ Доверие — сертификат подписан нашим CA и принят клиентом

2. КАК УСТАНАВЛИВАЕТСЯ HTTPS (TLS HANDSHAKE – КРАТКО)



Что получает клиент

- Сертификат сервера
- Цепочка CA
- Параметры шифрования

Что получает сервер

- Общий секрет для шифрования

3. ПУБЛИКАЦИЯ HTTPS НА СЕРВЕРЕ (ПРИМЕРЫ КОНФИГУРАЦИИ)



```
server {
    listen 443 ssl http2;
    server_name web01.company.test;

    ssl_certificate /etc/ssl/certs/web01.crt;
    ssl_certificate_key /etc/ssl/private/web01.key;
    ssl_trusted_certificate /etc/ssl/certs/ca_chain.crt;

    ssl_protocols TLSv1.2 TLSv1.3;
    ssl_ciphers HIGH:!aNULL:!MD5;
    ssl_prefer_server_ciphers on;

    location / {
        root /var/www/html;
        index index.html;
    }
}
```



```
<VirtualHost *:443>
    ServerName web01.company.test

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/web01.crt
    SSLCertificateKeyFile /etc/ssl/private/web01.key
    SSLCertificateChainFile /etc/ssl/certs/ca_chain.crt;

    SSLProtocol all -SSLv2 -SSLv3
    SSLCipherSuite HIGH:!aNULL:!MD5

    DocumentRoot /var/www/html
</VirtualHost>
```



Важно
После установки сертификатов перезапустите службу:
sudo systemctl reload nginx или sudo systemctl reload httpd

4. ДОВЕРИЕ КЛИЕНТА: УСТАНОВКА КОРНЕВОГО СЕРТИФИКАТА СА

Linux (Debian/Ubuntu)

Скопируйте корневой сертификат CA:
/usr/local/share/ca-certificates/company_rootCA.crt
и выполните:

```
sudo update-ca-certificates
```

Windows

Откройте company_rootCA.crt и установите:

- Локальный компьютер
- Поместите все сертификаты в следующее хранилище
- Доверенные корневые центры сертификации

После установки – перезапустите браузер.

Сертификат появится в системном хранилище и будет доверяться всеми приложениями.

5. ПРОВЕРКА КЛИЕНТА

БРАУЗЕР

https://web01.company.test

Соединение защищено
Сертификат действителен.

- ✓ Имя соответствует (SAN)
- ✓ Цепочка доверия валидна
- ✓ Срок действия корректен

OPENSSL (CLI)

```
$ openssl s_client -connect web01.company.test:443 \
-serrname web01.company.test -showcerts -verify 5

Verify return code: 0 (ok)
---
SSL-Session:
    Protocol : TLSv1.3
    Cipher : TLS_AES_256_GCM_SHA384
    Verify return code: 0 (ok)
```

ПО СЕТИ

```
$ curl -I https://web01.company.test

HTTP/2 200
server: nginx
content-type: text/html
strict-transport-security: max-age=31536000
...
```

Подсказка
Используйте -v в curl для подробной информации о TLS и сертификате.

6. ЧАСТЫЕ ПРОБЛЕМЫ И РЕШЕНИЯ

- | | |
|--|---|
| ⚠ Имя не совпадает (hostname mismatch) | Проверьте SAN и ServerName. Добавьте нужное имя в сертификат. |
| ⚠ Нет доверия к CA (unknown issuer) | Установите корневой сертификат CA в хранилище доверенных. |
| ⚠ Сертификат просрочен (certificate expired) | Проверьте срок действия и выпустите новый сертификат. |
| ⚠ Неполная цепочка (incomplete chain) | На сервере укажите полный файл цепочки (root + intermediate). |
| ⚠ OCSP/CRL недоступен | Проверьте доступность адресов CA. Используйте stapling (OCSP stapling). |

ЧЕК-ЛИСТ ПУБЛИКАЦИИ HTTPS

- ✓ Сертификат и ключ установлены на сервере
- ✓ Цепочка CA указана полностью
- ✓ TLS включён, старые протоколы отключены
- ✓ Служба перезапущена
- ✓ Клиент доверяет корневому сертификату CA
- ✓ Проверка: браузер, openssl, curl – без ошибок



РЕКОМЕНДАЦИИ ПО БЕЗОПАСНОСТИ

- Храните закрытый ключ с правами 600 и только на сервере
- Регулярно обновляйте сертификаты (до истечения срока)
- Используйте TLS 1.2 или 1.3, отключайте устаревшее
- Включайте HSTS и современные шифры
- Ведите журнал выпусков и проверок сертификатов



ПОЛЕЗНЫЕ ФАЙЛЫ

- | | |
|--------------------|--------------------------------------|
| web01.crt | – сертификат сервера |
| web01.key | – закрытый ключ сервера |
| ca_chain.crt | – цепочка (intermediate + root CA) |
| company_rootCA.crt | – корневой сертификат (для клиентов) |

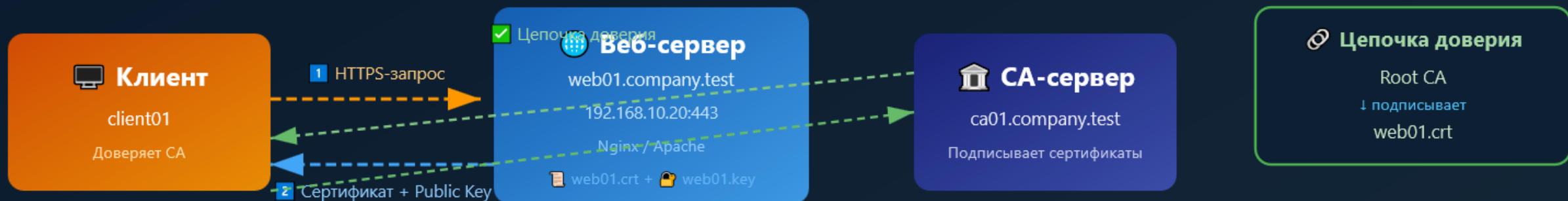


ИТОГ

HTTPS защищает данные, подтверждает подлинность сервера и строится на доверии к вашему CA.
Правильная публикация и доверие клиента – залог безопасной работы сервисов.

Публикация HTTPS-сервиса и доверие клиента

16:9



Настройка Nginx (HTTPS)

```
server {  
    listen 443 ssl;  
    server_name web01.company.test;  
    ssl_certificate /etc/ssl/certs/web01.crt;  
    ssl_certificate_key /etc/ssl/private/web01.key;  
}
```

Ключевые параметры

- ◆ listen 443 ssl — активация HTTPS
- ◆ server_name — должно совпадать с SAN
- ◆ ssl_certificate — публичный сертификат
- ◆ ssl_certificate_key — секретный ключ

→ HTTPS-запрос → Сертификат → Проверка доверия

Установка доверия на клиенте (Linux)

Шаг 1: Копирование CA

```
sudo cp ca.crt  
/usr/local/share/ca-certificates/  
→ company-test-root-ca.crt
```

Шаг 2: Обновление хранилища

```
sudo update-ca-certificates  
Обновляет доверенное хранилище  
✓ CA добавлен в систему
```

Шаг 3: Проверка

```
curl --cacert ca.crt  
https://web01.company.test  
✓ Без ошибок доверия!
```

8. Диагностика, проверка и типовые ошибки

Диагностика TLS начинается с локальной проверки файлов, затем переходит к проверке службы и клиента

Сначала проверяют сертификат, SAN, срок действия и соответствие закрытого ключа

Затем проверяют конфигурацию Nginx и доступность TCP-порта 443

После этого проверяют цепочку доверия со стороны клиента

Локальная проверка сертификата

`openssl x509 -in web01.crt -text -noout` показывает полное содержимое сертификата

В разделе Subject нужно увидеть
CN=web01.company.test

В разделе X509v3 Subject Alternative Name нужно увидеть DNS:web01.company.test и IP
Address:192.168.10.20

В разделе Validity проверяют Not Before и Not After, чтобы исключить истекший сертификат

Сетевое тестирование TLS-порта

`openssl s_client -connect web01.company.test:443 -CAfile ca.crt -servername web01.company.test` проверяет TLS-соединение

Параметр `-connect` задает сервер и порт, к которым подключается клиент

Параметр `-CAfile` указывает корневой сертификат СА для проверки доверия

SNI (Server Name Indication) передает имя сервера внутри TLS-подключения

Параметр `-servername` передает SNI-имя, чтобы сервер выбрал правильный виртуальный хост

Ожидаемый успешный признак: `Verify return code: 0 (ok)`

Проверка через curl

```
curl --cacert ca.crt https://web01.company.test
```

проверяет HTTPS как обычный клиент

Параметр `--cacert` явно указывает доверенный СА для этой проверки

Если команда работает с `--cacert`, но не работает без него, СА не импортирован в доверенное хранилище клиента

Если ошибка связана с именем, нужно проверить SAN и способ обращения к серверу

Ошибки имени и доверия

ERR_CERT_COMMON_NAME_INVALID означает, что имя в адресной строке не найдено в SAN сертификата

SEC_ERROR_UNKNOWN_ISSUER означает, что клиент не доверяет СА, который подписал сертификат

Self-signed certificate in certificate chain часто указывает на неполную или недоверенную цепочку

Правильное действие: проверить SAN, импорт СА и цепочку сертификатов

Ошибки срока действия и прав доступа

Expired certificate означает, что текущая дата вышла за пределы Not After

Certificate is not yet valid появляется, если системное время клиента или сервера настроено неверно

Permission denied при старте Nginx часто связан с правами на /etc/ssl/private/web01.key

Key values mismatch означает, что сертификат и закрытый ключ не образуют пару

Диагностика TLS по уровням

16:9

Уровень 1

Локальная проверка

```
openssl x509 -in cert.crt -text
```

```
openssl rsa -in key.key -check
```

```
openssl x509 -noout -modulus
```

✓ Проверка: SAN, срок, ключ

Уровень 2

Проверка службы

```
nginx -t # проверка синтаксиса
```

```
ss -tulpn | grep ':443'
```

```
systemctl reload nginx
```

✓ Права доступа к ключу

Уровень 3

Сетевая проверка

openssl s_client

```
-connect server:443
```

```
-CAfile ca.crt -servername
```

✓ Успешный признак:

```
Verify return code: 0 (ok)
```

✗ Ошибки сети:

- Connection refused
- Certificate chain incomplete

🔍 Проверка SNI

- Правильное имя в -servername
- Виртуальный хост на сервере
- Совпадение с SAN

🔗 Проверка цепочки

- Все промежуточные сертификаты
- Правильный порядок

Уровень 4

Клиентская проверка

curl --cacert ca.crt

```
https://server.company.test
```

✓ Браузер:

🔒 Замок → Безопасно

📁 Импорт CA:

```
/usr/local/share/ca-certificates/  
update-ca-certificates
```

✗ Типовые ошибки:

- ERR_CERT_COMMON_NAME_INVALID
- SEC_ERROR_UNKNOWN_ISSUER
- Self-signed certificate
- Certificate expired

Типовые ошибки

и их решения

ERR_CERT_COMMON_NAME_INVALID

- Проверить SAN
- Добавить DNS/IP в сертификат

SEC_ERROR_UNKNOWN_ISSUER

- Импортировать CA на клиент
- Проверить цепочку

Certificate expired

- Проверить дату/время
- Перевыпустить сертификат

Permission denied

- `chmod 600 /etc/ssl/private/`
- Проверить владельца

Key values mismatch

- Проверить modulus
- Перевыпустить с правильным ключом

Итоги лекции

- TLS защищает канал связи и помогает клиенту проверить подлинность сервера
- PKI строится на доверии к СА, цепочке сертификатов и защите закрытых ключей
- CSR не является секретом, а закрытые ключи СА и сервера должны храниться строго защищенно
- Современный серверный сертификат должен содержать корректный SAN
- Проверка TLS включает сертификат, SAN, цепочку доверия, права на ключ и сетевой порт службы

Контрольные вопросы

1. Почему HTTP, LDAP, SMTP и IMAP без TLS опасны для предприятия?
2. Чем симметричное шифрование отличается от асимметричного?
3. Зачем нужен цифровой сертификат, если у сервера уже есть пара ключей?
4. Какую роль выполняет CA в цепочке доверия?
5. Что такое CSR и почему его можно передавать на CA?
6. Почему современные браузеры требуют SAN в сертификате?
7. Какие файлы нельзя публиковать или передавать клиентам?
8. Что проверить при ошибке
ERR_CERT_COMMON_NAME_INVALID?
9. Как команда openssl s_client помогает проверить HTTPS-сервис?